

LAB 7

Types of Inheritance in C++

Multiple, Hierarchical, and Multilevel Inheritance

Learning Objectives

By the end of this lab, you should be able to:

- Distinguish between multiple, hierarchical, and multilevel inheritance.
- Draw the class diagram for each pattern from a one-line description.
- Write a class that inherits from two or more base classes (multiple inheritance).
- Write several classes that all derive from one common base (hierarchical inheritance).
- Build a chain of classes where each level extends the previous one (multilevel inheritance).
- Chain constructors correctly across all three patterns so every base is initialized.
- Recognize which pattern fits a given real-world problem.

1. The Three Patterns at a Glance

Last lab you saw the simplest case: one derived class inherits from one base class. In real programs, the relationships are usually richer. C++ supports three additional patterns, each useful for a different kind of "is-a" relationship.

 **Multiple Inheritance** — one class, several parents



 **Hierarchical Inheritance** — one parent, several children



 **Multilevel Inheritance** — a chain of classes

A
|
B
|
C



Quick way to remember which is which

Multiple — many parents.

Hierarchical — many children.

Multilevel — many levels (grandparent → parent → child).

2. Multiple Inheritance

Multiple inheritance occurs when a single class inherits from more than one base class. The derived class combines properties and behaviors from each parent.

Syntax

```
class Derived : public Base1, public Base2 {
    // members of both Base1 AND Base2 are available
};
```

Key Points

- A derived class can access members of all its base classes.
- If two base classes have a member with the same name, accessing it is ambiguous — the compiler will not know which one you mean.
- Resolve ambiguity using the scope resolution operator: *Base1::member* or *Base2::member*.

Code 1 — Combining Two Independent Classes

Goal: see how class C automatically gains the functions of both A and B.

The Code

```
#include <iostream>
using namespace std;

class A {
public:
    void showA() {
        cout << "This is class A" << endl;
    }
};

class B {
```

```

public:
    void showB() {
        cout << "This is class B" << endl;
    }
};

class C : public A, public B {      // <-- inherits from BOTH A and B
public:
    void showC() {
        cout << "This is class C" << endl;
    }
};

int main() {
    C obj;
    obj.showA();      // inherited from A
    obj.showB();      // inherited from B
    obj.showC();      // own function
    return 0;
}

```

Code Breakdown

Code	What it does
class A { ... }; class B { ... };	← Two completely independent base classes — neither knows about the other.
class C : public A, public B	← C inherits from BOTH. Note the comma-separated list of parents.
obj.showA();	← Works — showA was inherited from A.
obj.showB();	← Works — showB was inherited from B.
obj.showC();	← Works — showC was defined in C itself.

Expected Output

► Output

```

This is class A
This is class B
This is class C

```

Code 2 — Multiple Inheritance with Constructors

Goal: a more realistic example where both parents have constructors that need to be initialized.

The Code

```

#include <iostream>
using namespace std;

```

```

class Student {
protected:
    string name;
public:
    Student(string n) {
        name = n;
        cout << "Student constructor called" << endl;
    }
};

class Athlete {
protected:
    string sport;
public:
    Athlete(string s) {
        sport = s;
        cout << "Athlete constructor called" << endl;
    }
};

class ScholarAthlete : public Student, public Athlete {
    int score;
public:
    ScholarAthlete(string n, string s, int sc)
        : Student(n), Athlete(s) {          // initialize BOTH bases
        score = sc;
        cout << "ScholarAthlete constructor called" << endl;
    }
    void display() {
        cout << "Name: " << name
            << ", Sport: " << sport
            << ", Score: " << score << endl;
    }
};

int main() {
    ScholarAthlete s("Ali", "Football", 90);
    s.display();
}

```

Code Breakdown

Code	What it does
<code>class ScholarAthlete : public Student, public Athlete</code>	← Inherits from BOTH Student and Athlete.
<code>: Student(n), Athlete(s)</code>	← Constructor chaining for TWO bases — separated by commas, both in the initializer list.
<code>cout << name << sport << score</code>	← ScholarAthlete uses name from Student, sport from Athlete, and its own score.

Code	What it does
<code>ScholarAthlete s("Ali", "Football", 90);</code>	← Triggers Student, then Athlete, then ScholarAthlete constructors — in that order.

Expected Output

► Output

```
Student constructor called
Athlete constructor called
ScholarAthlete constructor called
Name: Ali, Sport: Football, Score: 90
```



Construction order follows the inheritance list

The base classes are constructed in the order they appear in the **class declaration**, NOT the order in the initializer list. *public Student, public Athlete* → Student first, Athlete second. Try swapping them and watch the output change.



The diamond problem (preview)

If both Student and Athlete themselves inherited from a common class (say, Person), ScholarAthlete would end up with two copies of that Person. This is the famous "diamond problem" and is solved using virtual inheritance — a topic for a future lab. For now, keep your multiple-inheritance bases independent.

3. Hierarchical Inheritance

Hierarchical inheritance is the opposite arrangement: one base class, multiple derived classes. Every derived class shares the base's functionality but adds its own specialization.

Syntax

```
class Base { };  
class Derived1 : public Base { };  
class Derived2 : public Base { };
```

Key Points

- One base class, multiple derived classes.
- Promotes code reuse — the shared functionality lives in one place.
- Each derived class can add its own unique data and behavior on top of the inherited base.

Code 1 — One Base, Two Children

Goal: classes B and C both inherit from A. Each one can call show() because it came from A, and each one also has its own unique function.

The Code

```
#include <iostream>  
using namespace std;  
  
class A {  
public:  
    void show() {  
        cout << "This is base class A" << endl;  
    }  
};  
  
class B : public A {  
public:  
    void displayB() {  
        cout << "This is class B" << endl;  
    }  
};  
  
class C : public A {          // <-- also inherits from A, independently of B  
public:  
    void displayC() {  
        cout << "This is class C" << endl;  
    }  
};  
  
int main() {  
    B obj1;
```

```

    C obj2;

    obj1.show();      // inherited from A
    obj1.displayB();  // own function

    obj2.show();      // inherited from A
    obj2.displayC();  // own function

    return 0;
}

```

Code Breakdown

Code	What it does
<code>class A { void show() {...} };</code>	← Base class with one shared function.
<code>class B : public A { ... }</code>	← B inherits <code>show()</code> from A AND adds its own <code>displayB()</code> .
<code>class C : public A { ... }</code>	← C ALSO inherits <code>show()</code> from A, completely independently of B. B and C are siblings, not parent-child.
<code>obj1.show(); obj2.show();</code>	← Both objects use the SAME inherited function. Write once, reuse many times.

Expected Output

► Output

```

This is base class A
This is class B
This is base class A
This is class C

```

Code 2 — Person with Teacher and Student Children

Goal: a real-world hierarchical example. Both Teacher and Student are kinds of Person, but they specialize in different ways.

The Code

```

#include <iostream>
using namespace std;

class Person {
protected:
    string name;
public:
    Person(string n) {

```

```

        name = n;
        cout << "Person constructor called" << endl;
    }
};

class Teacher : public Person {
    string subject;
public:
    Teacher(string n, string sub) : Person(n) {
        subject = sub;
        cout << "Teacher constructor called" << endl;
    }
    void display() {
        cout << "Teacher: " << name << ", Subject: " << subject << endl;
    }
};

class Student : public Person {
    int id;
public:
    Student(string n, int i) : Person(n) {
        id = i;
        cout << "Student constructor called" << endl;
    }
    void display() {
        cout << "Student: " << name << ", ID: " << id << endl;
    }
};

int main() {
    Teacher t("Sara", "C++");
    Student s("Rahim", 101);

    t.display();
    s.display();
    return 0;
}

```

Code Breakdown

Code	What it does
class Person { protected: string name; ... }	← Shared base with a single field name.
class Teacher : public Person	← Teacher adds a subject field of its own.
class Student : public Person	← Student adds an id field — different from Teacher's specialization.
: Person(n)	← Both derived constructors chain to Person(n), so name gets initialized exactly once per object.

Code	What it does
Both have a <code>display()</code> function	← Same name, different content. Each class is free to format its own output.

Expected Output

► Output

```
Person constructor called
Teacher constructor called
Person constructor called
Student constructor called
Teacher: Sara, Subject: C++
Student: Rahim, ID: 101
```



Why "Person constructor called" appears twice

Each new object has its OWN copy of name and its OWN Person sub-object. Creating Teacher `t` and Student `s` independently means Person's constructor runs once for each — exactly what we want.

4. Multilevel Inheritance

Multilevel inheritance forms a chain: $A \rightarrow B \rightarrow C$. Class B inherits from A, then class C inherits from B. C ends up with everything from both A AND B.

Syntax

```
class A { };
class B : public A { };
class C : public B { };
```

Key Points

- Inheritance happens in levels — grandparent, parent, child.
- The deepest derived class inherits properties of ALL classes above it in the chain.
- Promotes step-by-step specialization — each level adds a bit more detail.

Code 1 — A Three-Level Chain

Goal: an object of type C should be able to call showA, showB, AND showC.

The Code

```
#include <iostream>
using namespace std;

class A {
public:
    void showA() { cout << "Class A" << endl; }
};

class B : public A {          // B inherits from A
public:
    void showB() { cout << "Class B" << endl; }
};

class C : public B {          // C inherits from B (and indirectly from A)
public:
    void showC() { cout << "Class C" << endl; }
};

int main() {
    C obj;
    obj.showA();    // inherited from A (through B)
    obj.showB();    // inherited from B
    obj.showC();    // own function
    return 0;
}
```

Code Breakdown

Code	What it does
<code>class B : public A</code>	← Level 2 — B is a direct child of A.
<code>class C : public B</code>	← Level 3 — C is a direct child of B and an INDIRECT child of A.
<code>obj.showA();</code>	← Works! C inherited <code>showA()</code> from A through B. Functions "pass down" the entire chain.
<code>obj.showB();</code>	← Works — directly inherited from B.
<code>obj.showC();</code>	← Works — defined in C itself.

Expected Output

► Output

```
Class A
Class B
Class C
```

Code 2 — Person → Employee → Manager

Goal: a realistic three-level hierarchy. A Manager IS an Employee, who IS a Person. Each level adds more detail, and the Manager constructor must chain all the way up.

The Code

```
#include <iostream>
using namespace std;

class Person {
protected:
    string name;
public:
    Person(string n) {
        name = n;
        cout << "Person constructor called" << endl;
    }
};

class Employee : public Person {
protected:
    int salary;
public:
    Employee(string n, int s) : Person(n) {          // chain up to Person
        salary = s;
        cout << "Employee constructor called" << endl;
    }
};

class Manager : public Employee {
```

```

    string department;
public:
    Manager(string n, int s, string d) : Employee(n, s) {    // chain up to Employee
        department = d;
        cout << "Manager constructor called" << endl;
    }
    void display() {
        cout << "Name: " << name
            << ", Salary: " << salary
            << ", Department: " << department << endl;
    }
};

int main() {
    Manager m("Karim", 50000, "IT");
    m.display();
    return 0;
}

```

Code Breakdown

Code	What it does
Person → Employee → Manager	← A 3-level chain. Manager indirectly inherits everything from Person.
Employee(...) : Person(n)	← Employee's constructor chains UP to Person — sets name.
Manager(...) : Employee(n, s)	← Manager chains UP to Employee — which itself chains up to Person. The whole chain runs.
cout << name << salary << department	← Manager can read name (from Person), salary (from Employee), AND its own department in one statement.

Expected Output

► Output

```

Person constructor called
Employee constructor called
Manager constructor called
Name: Karim, Salary: 50000, Department: IT

```



Constructor execution order — top to bottom

In multilevel inheritance, constructors fire from the TOP of the chain downward: Person first, then Employee, then Manager. Destructors fire in REVERSE: Manager first, then Employee, then Person. Same rule as Lab 5 — "last built, first destroyed."



Each level must chain to the level directly above

Manager cannot skip Employee and call Person(n) directly. The chain has to be: Manager → Employee → Person. If Employee does not chain to Person, the Person sub-object will never be initialized properly.

5. Quick Comparison

All three patterns are forms of single-direction inheritance, but they differ in HOW the classes are connected. This table sums it all up:

Pattern	Shape	Typical use case
Multiple	Many parents → one child	Combining unrelated capabilities (e.g., a class that is BOTH a Student and an Athlete).
Hierarchical	One parent → many children	Several specializations of one general concept (e.g., Person → Teacher, Student, Doctor).
Multilevel	Grandparent → Parent → Child (chain)	Step-by-step refinement (e.g., Person → Employee → Manager).

6. Summary

Five things to take away from this lab:

- **Multiple inheritance lets a class have several parents.** Use a comma-separated list and chain to each base in the initializer list.
- **Hierarchical inheritance lets one parent have many children.** Shared code lives in the base; each child specializes independently.
- **Multilevel inheritance forms a chain.** The deepest child gets everything from every level above it.
- **Constructors always run base → derived.** Each derived constructor must chain to its immediate base if that base has no default constructor.
- **Pick the pattern that fits the real-world relationship.** If you find yourself fighting the inheritance shape, you are probably using the wrong pattern.

7. Practice Tasks

Task 1 — Multiple Inheritance: Person + Employee → Manager

Create the following:

- **Base class:** Person — has *name*.
- **Base class:** Employee — has *salary*.
- **Derived class:** Manager — inherits from BOTH Person and Employee, adds *department*.

Requirements

- Each class has a constructor that prints a message when it runs.
- Manager's constructor must chain to BOTH Person and Employee.
- Provide a `display()` function that prints name, salary, and department in one line.

- In main, create at least one Manager and call display().

Hint for Task 1

The Manager constructor signature should be something like *Manager(string n, int s, string d) : Person(n), Employee(s) { ... }*. Observe in the output that both base constructors run before Manager's does.

Task 2 — Hierarchical Inheritance: Vehicle → Car, Bike

Create the following:

- **Base class:** Vehicle — has *brand*.
- **Derived class:** Car — adds *number of doors*.
- **Derived class:** Bike — adds *engine capacity*.

Requirements

- Use constructors to initialize all data members.
- Create one Car object and one Bike object in main.
- Each derived class has its own display() that shows all of its details (including brand from the base).
- Notice how the brand field is reused from Vehicle — that is the whole point of code reuse.

Hint for Task 2

Make brand protected (not private) so Car and Bike can read it directly in their display functions. Alternatively, give Vehicle a showBrand() helper that the children can call.

Task 3 — Multilevel Inheritance: Person → Student → GraduateStudent

Create the following:

- **Base class:** Person — has *name*.
- **Derived class:** Student — adds *ID*.
- **Derived class:** GraduateStudent — adds *research topic*.

Requirements

- Each level has its own constructor that prints a message.
- Values must be passed all the way up through constructor chaining — GraduateStudent → Student → Person.
- GraduateStudent's display() should print name (from Person), ID (from Student), and research topic (its own) in one go.
- Create at least one GraduateStudent object in main and observe the order in which the three constructors run.



What you should observe

Three constructor-called messages should print in order: Person first, then Student, then GraduateStudent. This proves data flowed correctly across all three levels — and that the C++ language built the object from the top down.

Prepared by

Alifa Shanzidah Manarat

Lecturer

Dept. of CSE, BAUST